Computational Geometry: Theory and Applications 2 (1992) 1–14
Elsevier

Relative neighborhood graphs in three dimensions*

Pankaj K. Agarwal

Computer Science Department, Duke University, Durham, NC 27706, USA

Jiří Matoušek

Department of Applied Mathematics, Charles University, Praha, Czechoslovakia

Communicated by Herbert Edelsbrunner
Submitted 1 May 1991
Accepted 23 September 1991

Abstract

Agarwal, P.K. and J. Matoušek, Relative neighborhood graphs in three dimensions, Computational Geometry: Theory and Applications 2 (1992) 1–14.

The relative neighborhood graph (RNG) of a set S of n points in $\mathbb{R}^d$ is a graph (S, E), where $(p, q) \in E$ if and only if there is no point $z \in S$ such that $\max\{d(p, z), d(q, z)\} < d(p, q)$. We show that in $\mathbb{R}^3$, RNG(S) has $O(n^{4/3})$ edges. We present a randomized algorithm that constructs RNG(S) in expected time $O(n^{3/2+\varepsilon})$ assuming that the points of S are in general position. If the points of S are arbitrary, the expected running time is $O(n^{7/4+\varepsilon})$. These algorithms can be made deterministic without affecting their asymptotic running time.

Keywords. Pattern matching; geometric graphs; arrangements; random sampling; closest pairs.

1. Introduction

Let S be a set of n points in $\mathbb{R}^d$. The *relative neighborhood graph* of S, denoted RNG(S), is a graph (S, E), where a pair of points $(p, q) \in E$ if and only if

$$d(p, q) \leq \max_{p' \in S - \{p,q\}} \{d(p, p'), d(q, p')\}.$$

Here $d(.,.)$ is the Euclidean distance. In other words, (p, q) is an edge of RNG(S) if the interior of the *lune* of p and q, defined as the set of points

$$\lambda(p, q) = \{z \mid d(p, z) \leq d(p, q) \text{ and } d(q, z) \leq d(p, q)\},$$

does not contain any point of S.

* Work by the first author was supported by National Science Foundation Grant CCR-91-06514. Part of this work was done while the second author was visiting School of Mathematics, Georgia Institute of Technology, Atlanta.

Relative neighborhood graphs were originally introduced by Toussaint [23], and have applications in pattern recognition. See [12, 15, 17, 19–21, 24] for variants and generalizations of relative neighborhood graphs.

It is well known that RNG(S) is a subgraph of Delaunay triangulation of S, therefore RNG(S) is a planar graph in $\mathbb{R}^2$. But, for $d \geq 3$, Delaunay triangulation can have $\Omega(n^2)$ edges in the worst case, so an interesting question is to obtain a tight bound on the size of RNG(S). It turns out that for $d \geq 4$, RNG(S) can also have $\Omega(n^2)$ edges [14]. But for $d = 3$, an upper bound of $O(n^{3/2}\beta(n))$ has been proved by Jaromczyk and Kowaluk [14], where $\beta(n)$ is an extremely slowly growing function. In this paper we improve the upper bound to $O(n^{4/3})$ by reducing it to counting the number of bi-chromatic closest pairs. Note that if we assume points in $\mathbb{R}^d$ to be in general position, that is, only $O(1)$ points lie on a $(d-1)$-sphere, then RNG(S) has only linear number of edges; see e.g. [22].

Like minimum spanning tree, a common approach for computing RNG(S) is—first construct a graph G that contains all edges of RNG(S) and then throw away the edges of G that do not appear in RNG(S). In $\mathbb{R}^2$, Delaunay triangulation can be used as G, and therefore RNG(S) can be computed in $O(n \log n)$ time [22, 16, 25]. But for $d = 3$, one has to use some other graph if one wants to construct it in subquadratic time, because, as mentioned above, Delaunay triangulation can have quadratic number of edges. The previously best known algorithm for constructing RNG(S) in $\mathbb{R}^3$ is by Jaromczyk and Kowaluk, whose time complexity is $O(n^2 \log n)$. If points are in general position, the running time can be improved to $O(n^2)$ [13, 22].

In this paper, we present a randomized algorithm whose expected running time is $O(n^{3/2+\varepsilon})$, assuming that only constant number of points lie on a sphere.[1] It can be extended to higher dimensions too. If the points in S are arbitrary, then RNG(S) can be computed in $\mathbb{R}^2$ in expected time $O(n^{7/4+\varepsilon})$ by modifying the previous algorithm. Both of these algorithms can be made deterministic without affecting their asymptotic time complexity.

We extend our algorithms to compute k-relative neighborhood graphs. k-RNG(S) has an edge (p, q) if the interior of $\lambda(p, q)$ contains less than k points of S.

This paper is organized as follows. In Section 2 we prove the upper bound on RNG in $\mathbb{R}^3$. Section 3 gives the main algorithm. We generalize our algorithm to arbitrary set of points in Section 4, and to k-relative neighborhood graphs in Section 5. We conclude with some final remarks in Section 6.

2. Complexity of RNG

In this section we show that RNG(S) of a set S of n points in $\mathbb{R}^3$ has $O(n^{4/3})$

[1] Throughout this paper, ε denotes an arbitrarily small positive constant, and the constant of proportionality in the time complexity tends to ∞ as $\varepsilon \downarrow 0$.

edges. In order to prove the bound, we need to define a few notation, some of which are borrowed from [1]. Throughout this section we shall not distinguish between points and vectors. Given a unit vector $u \in \mathbb{R}^d$ and an angle α, let

$$\text{Cone}(u, \alpha) = \{x \in \mathbb{R}^3 \mid \angle(x, u) \leq \alpha\},$$

where

$$\angle(x, u) = \arccos \frac{x^T u}{\|x\| \cdot \|u\|}.$$

Let P, Q be two sets of points. The pair (P, Q) is called *well separated* if there are a point $z \in \mathbb{R}^3$ and a unit vector u, such that $P \subset z + \text{Cone}(-u, \alpha)$ and $Q \subset z + \text{Cone}(u, \alpha)$ for some $\alpha < \pi/6$ (see Fig. 1). The edges of $\text{RNG}(P \cup Q)$ of the form $(p, q) \in P \times Q$ will be referred to as *cross edges*. A point $q \in Q$ is called a *bi-chromatic closest neighbor* of $p \in P$ if $d(p, q) = \min_{q' \in Q} d(p, q')$, a pair $(p, q) \in P \times Q$ is called a *bi-chromatic closest pair* if $d(p, q) = \min_{p' \in P, q' \in Q} d(p', q')$, and a pair (p, q) is called a *symmetric bi-chromatic closest neighbor pair* if q is a bi-chromatic closest neighbor of p and vice-versa.

Lemma 2.1. *Let P, Q be a well separated pair of set of points in $\mathbb{R}^d$. Then $\text{RNG}(P \cup Q)$ has a cross edge (p, q) if and only if (p, q) is bi-chromatic closest neighbor pair.*

Proof. Let (p, q) be a bi-chromatic closest neighbor pair. Then the ball B_p (resp. B_q) of radius $d(p, q)$ around p (resp. q) does not contain any point of P (resp. Q) in its interior. Consequently, lune $\lambda(p, q) = B_p \cap B_q$ does not contain any point of $P \cup Q$ in its interior. Therefore, (p, q) is an edge in $\text{RNG}(P \cup Q)$.

For the 'only if' part, assume, in order to obtain a contradiction, that q is not a bi-chromatic closest neighbor of p and $(p, q) \in \text{RNG}(P \cup Q)$, i.e., there is a point $q' \in Q$ such that $d(p, q') < d(p, q)$ and q' does not lie in the interior of $\lambda(p, q)$.

Let h be the plane passing through p, q and q'. Let s be the intersection point of h and the boundary of $\lambda(p, q)$ such that $d(p, q) = d(q, s) = d(p, s)$ and that s and q' lie on the same side of the line supporting pq (see Fig. 2). Let ρ_1 (resp. ρ_2) denote the ray emanating from p and passing through q (resp. s). The angle of the wedge formed by ρ_1 and ρ_2 is $\pi/3$, as $\triangle spq$ is an equilateral triangle. Since

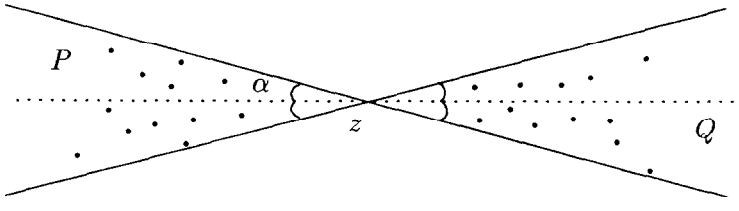

Fig. 1. A well separated pair of sets.

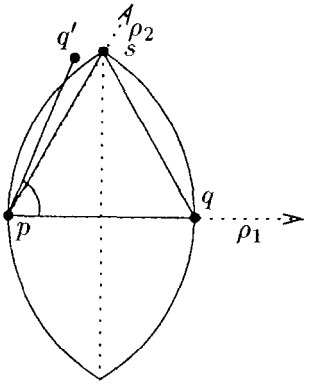

Fig. 2. Triangle *spq*.

$q' \notin \lambda(p, q)$ and $d(p, q) > d(p, q')$, it is easily seen that q' cannot lie in this wedge, therefore $\angle qpq' \geq \pi/3$. But that is impossible, because $\angle qpq' \leq 2\alpha < \pi/3$. Hence, q is a bi-chromatic closest neighbor of p.

Similarly one can prove that p is a bi-chromatic closest neighbor of q. □

Lemma 2.2. *Let P, Q be a well separated pair of set of points in $\mathbb{R}^3$. Then $\mathrm{RNG}(P \cup Q)$ has $O(m^{2/3}n^{2/3} + m + n)$ cross edges, where $m = |P|$ and $n = |Q|$.*

Proof. Let G be the subgraph of $\mathrm{RNG}(P \cup Q)$ induced by its cross edges. Let $G_1, \ldots, G_t$ denote the nontrivial connected components of G, and let (P_i, Q_i) denote the vertices of G_i. We claim that each edge (p, q) in (P_i, Q_i) connects a bi-chromatic closest pair of P_i an Q_i. Suppose the contrary, i.e., there are a bi-chromatic closest pair $(p'q')$ of (P_i, Q_i) and an edge (p, q) in G_i such that $d(p, q) > d(p', q')$. Since G_i is a connected graph, one of the following two conditions should hold: (i) there is a vertex $z \in P_i \cup Q_i$ such that two of the edges incident to z have different lengths, or (ii) length of all edges of G_i is $d(p, q)$. But (i) violates Lemma 2.1 for z and (ii) violates the same lemma for p'. Hence, all the edges of G_i connect bi-chromatic closest pairs of (P_i, Q_i).

Edelsbrunner and Sharir [11] have proved that the number of bi-chromatic closest pairs between a set of a points and another set of b points in $\mathbb{R}^3$ is $O(a^{2/3}b^{2/3} + a + b)$. Thus, the number of edges in G_i is $O(m_i^{2/3}n_i^{2/3} + m_i + n_i)$, where $m_i = |P_i|$, $n_i = |Q_i|$. Summing it over all connected components of G, we obtain the desired bound. □

The argument of Lemma 2.2 also implies the following.

Corollary 2.3. *The number of symmetric bi-chromatic closest neighbor pairs between a set of m points and another set of n points in $\mathbb{R}^3$ is $O(m^{2/3}n^{2/3} + m + n)$.*

To prove the main result of this section, we need a procedure that decomposes $S \subset \mathbb{R}^d$ into a family

$$\mathcal{F} = \{(P_1, Q_1), (P_2, Q_2), \ldots, (P_s, Q_s)\}$$

of well-separated pairs with the following properties:

(1) $\sum_{i=1}^{s} (|P_i| + |Q_i|) = O(n \log^{d-1} n)$, and

(2) for every pair $p, q \in S$, there is an $i \leq s$ such that $p \in P_i$ and $q \in Q_i$.

The second condition ensures that every edge of RNG(S) appears as a cross edge in at least one of RNG($P_i \cup Q_i$). Agarwal et al. [1] have given a procedure to construct such a family of pairs of sets in $\mathbb{R}^d$. For the sake of completeness, we shall describe it briefly here.

A basis $B = \{b_1, \ldots, b_d\}$ of $\mathbb{R}^d$ is called *narrow* if there exist a unit vector u and an angle $\alpha < \pi/3$ such that

$$\text{cone}(B) = \left\{ \sum_{i=1}^{d} \lambda_i b_i \mid \lambda_i \geq 0, \forall i \right\} \subseteq \text{Cone}(u, \alpha).$$

Let $\mathcal{F}$ be a family of $t = O(1)$ narrow bases such that $\bigcup_{B \in \mathcal{F}} (\text{cone}(B) \cup \text{cone}(-B)) = \mathbb{R}^d$. Yao [26] has shown that for every dimension d one can compute such a family of narrow bases in $O(1)$ time. For each $B \in \mathcal{F}$, we compute a set of well-separated pairs of subsets of S. Let $(x_1, \ldots, x_d)$ be the coordinates of a point of x in basis B, i.e., $x = x_1 b_1 + x_2 b_2 + \cdots + x_d b_d$. Here is the outline of the algorithm. Initially $k = d$ and $P = Q = S$.

We repeat the above procedure for all $B \in \mathcal{F}$. It has been shown in [1] that the above procedure returns a family of well separated pairs that satisfy (i) and (ii). Let $E_k(m+n)$ denote the number of cross-edges in the pairs of sets returned by the algorithm Pairing(k, P, Q) (see Fig. 3) with $|P| = m$, $|Q| = n$, then

$$E_k(m+n) \leq 2E_k\left(\frac{m+n}{2}\right) + E_{k-1}(m+n) \tag{2.1}$$

Algorithm: Pairing (k, P, Q)
if $k = 0$ and $P, Q \neq \emptyset$ **then**
  output (P, Q)
end if
if $k \geq 1$ **then**
  $med_k :=$ median of the kth coordinate of $P \cup Q$
  $P_l = \{p \in P \mid p_k \leq med_k\}$, $P_r = \{p \in P \mid p_k > med_k\}$
  $Q_l = \{p \in Q \mid p_k \leq med_k\}$, $Q_r = \{p \in Q \mid p_k > med_k\}$
  Pairing (k, P_l, Q_l), Pairing (k, P_r, Q_r)
  Pairing ($k-1$, P_l, Q_r)
end if

Fig. 3. Computing the family of well separated pairs.

and, by Lemma 2.2, $E_0(m+n) = O(m^{2/3}n^{2/3} + m + n)$. The solution of the above recurrence is easily seen to be $O(m^{2/3}n^{2/3} + (m+n)\log^k(m+n))$. Initially $P = Q = S$ and $t = O(1)$, therefore we can conclude the following.

Theorem 2.4. *Given a set S of n points in $\mathbb{R}^3$, RNG(S) has $O(n^{4/3})$ edges.*

Remark 2.5. (i) It is easy to show that a lower bound on the number of bi-chromatic closest pairs will yield a similar lower bound on the size of relative neighborhood graphs.

(ii) If we assume that the points of S are in general position, that is, no $d+2$ points lie on a $(d-1)$-sphere, the size of RNG(S) is obviously linear (see e.g. [22]).

3. Computing RNG: points in general position

In this section we assume that the point of $S \subset \mathbb{R}^3$ are in general position—no five points lie on a sphere. This condition implies that every point of S has only constant number of closest neighbors. RNG(S) is constructed in the following three steps.

I. Compute the family $\mathcal{F}$ of well separated pairs of subsets of S, as described above.
II. For each pair $(P, Q) \in \mathcal{F}$, compute a bi-chromatic closest neighbors of each $z \in P \cup Q$. Let $E_{P,Q} \subset P \times Q$ be the set of edges (p, q) such that p is the bi-chromatic closest neighbor of q and vice-versa, and let $E = \bigcup_{P,Q} E_{P,Q}$.
III. Throw away the edges of E that are not the edges of RNG(S).

Steps I and II can be accomplished together by replacing the first step of Pairing(k, P, Q) with

if $k = 0$ and $P, Q \neq \emptyset$ **then**
Compute the bi-chromatic closest neighbors of $z \in P \cup Q$

By our assumptions on points being in general position, each point in $P \cup Q$ has only constant number of bi-chromatic closest neighbors, therefore they can be computed either in time $O((m\sqrt{n} + n\sqrt{m})\log(m+n))$ using Voronoi diagrams, or in randomized expected time

$$O(m^{2/3}n^{2/3}\log^{4/3} m + m\log^2 n + n\log^2 n)$$

using a more sophisticated algorithm of Agarwal et al [3] (this algorithm extends also to higher dimensions). For our purposes, the first algorithm is sufficient. Analyzing in the same way as in the previous section, one can show that Steps I and II require $O(n^{3/2}\log n)$ time.

Recall that (p, q) is an edge of RNG(S) if the interior of $\lambda(p, q)$ does not contain a point of S. We thus need to solve the following problem. Given a set $\mathscr{L}$ of n lunes and a set S of m points, determine the empty lunes of $\mathscr{L}$, i.e., the lunes that do not contain any point of S.

We present an algorithm based on the random sampling technique. As in most of the other random-sampling based algorithms, we first describe an algorithm that works well when $n \gg m$, and then describe another algorithm, which uses the previous algorithm as a subroutine, and is efficient for all ranges of m and n.

First algorithm: The first algorithm constructs a two-level data structure, in time $O(m^{2+\varepsilon})$, on S so that, for a query lune, one can determine in $O(\log^2 n)$ time whether it contains any points of S in its interior. This gives an $O(m^{3+\varepsilon} + n \log^2 n)$ algorithm for filtering out the RNG edges from E. The running time can be improved to $O(mn^{2/3+\varepsilon} + n^{1+\varepsilon})$ by partitioning S into $m/n^{1/3}$ sets each of size $\leq \lceil n^{1/3} \rceil$ and running the above algorithm for each subset of S separately. The data structure is based on the partitioning scheme of Chazelle et al [6]. We shall only sketch the main idea; the details can be found in the original paper.

Let S^* be the set of planes dual to the points of S and let r be a suitable constant. We compute a set $\mathscr{T}$ of simplices with disjoint interiors, which cover the whole space and each of which intersects at most m/r planes of S^*. It is known that we can find such a collection of size $O(r^3)$ in expected linear time [5]. (One can find such a collection of size $O(r^3 \log^3 r)$ by a straightforward random sampling.) For each simplex $\tau \in \mathscr{T}$, let $S_\tau^* \subseteq S^*$ denote the set of at most m/r planes intersecting the interior of τ. We recursively construct the structure on S_τ^* and store it at τ. We also store two secondary structures at τ. Let U_τ (resp. L_τ) denote the set of points dual to the planes of S^* that lie above (resp. below) τ. We preprocess U_τ into a data structure, so that, for a query point, we can quickly determine its closest neighbor in U_τ. Clarkson [7] has shown that a set of t points in $\mathbb{R}^d$ can be preprocessed in time $O(t^{\lfloor d/2+\delta \rfloor})$, so that one can answer a closest neighbor query in $O(\log t)$ time. Therefore U_τ can be processed in time $O(|U_\tau|^{2+\delta})$ into a data structure that supports $O(\log m)$ time closest neighbor queries. We construct a similar structure for L_τ. Following the same analysis as in [6], one can show that the overall time and space required by this data structure is $O(m^{3+\varepsilon})$, for any $\varepsilon > 0$.

Let $\lambda(p, q)$ be a query lune, and let h_{pq} be the perpendicular bisector of p and q. Without loss of generality we can assume that p lies below h_{pq}. We search the primary structure with h_{pq}^*, point dual to h_{pq}, starting from the root. We first locate the simplex $\tau \in \mathscr{T}$ containing the point h_{pq}^*. We recursively search through the primary substructure stored at τ to determine whether any point of S_τ lies in the lune. Since all planes of U_τ^* lie above τ, and therefore above h_{pq}, $\lambda(p, q)$ contains a point of U_τ if and only if the ball B of radius $d(p, q)$, centered at p, contains a point of U_τ. In other words $\text{int}(\lambda(p, q)) \cap U_\tau \neq \emptyset$ if and only if the distance between p and its closest neighbor in U_τ is less than $d(p, q)$. We can

therefore determine, in $O(\log m)$ time, whether the interior of $\lambda(p, q)$ contains a point of U_τ using the secondary structure stored at τ. Similarly, one can determine whether any point of L_τ lies inside $\lambda(p, q)$. Since the query procedure visits $O(\log m)$ nodes of the primary structure, the total query time is $O(\log^2 m)$ as required. Hence, we can conclude the following.

Lemma 3.1. *Given a set S of m points in $\mathbb{R}^3$ and a collection $\mathscr{L}$ of n lunes, one can determine the subset of lunes that do not contain any point of S in time $O(mn^{2/3+\varepsilon} + n^{1+\varepsilon})$.*

Remark 3.2. Given a fixed $k = O(1)$, the algorithm can be modified to determine the subset of lunes that contain less than k points of S in their interior, as follows: At each node of the primary structure, we preprocess U_τ in such a way so that instead of just deciding the emptiness of B, we can decide whether the interior of the ball B contains at most k points of U_τ and, if yes, we can also count the number of points of U_τ that lie in B. To this end, we preprocess U_τ into a data structure of size $O(|U_\tau|^{2+\varepsilon})$, so that k closest neighbors of a query point can be determined in $O(k \log m)$ time; see [2]. One constructs a similar data structure for L_τ. We leave it for the reader to verify that with these data structures, one can determine in time $O(\log^2 m + k \log m)$ whether a lune contains $\geq k$ points. The overall running time is easily seen to be $O(mn^{2/3+\varepsilon} + n^{1+\varepsilon})$.

Second algorithm: Next, we describe the second algorithm for computing the subset of empty lunes of $\mathscr{L}$. The algorithm consists of the following steps.

1. If $m < n^{1/3}$, then solve the problem using the previous algorithm. Otherwise, do the following.
2. Randomly choose a subset $\mathscr{C} \subseteq \mathscr{L}$ of r lunes, where r is a sufficiently large constant.
3. Construct the arrangement of spheres bounding the lunes of $\mathscr{C}$ and decompose the arrangement into $O(r^3\beta(r))$ constant size cells, where $\beta(r)$ is an extremely slowly growing function. Let $\mathscr{A}(\mathscr{C})$ denote the resulting subdivision.
4. For each cell τ, determine the set $\mathscr{L}_\tau$ of lunes whose boundaries intersect τ, and the set $S_\tau \subseteq S$ of points that lie inside τ.
5. If $S_\tau \neq \emptyset$ for some τ, then, for all lunes λ that contain τ, conclude that λ is not empty. Discard these lunes from all $\mathscr{L}_\tau$.
6. If τ is a 3-dimensional cell, solve the problem recursively for $\mathscr{L}_\tau$, S_τ. If τ is a 2-dimensional cell, again solve the problem recursively using a 2-dimensional variant of the algorithm.
7. If τ is a 1-dimensional cell, we can easily determine all lunes of $\mathscr{L}_\tau$ that do not contain any point of S_τ in their interior: Sort the points of S_τ along τ and, for each lune $\lambda \in \mathscr{L}_\tau$, determine whether $\text{int}(\lambda) \cap \tau$ contains any point of S_τ. The second step can be easily done by a straight-forward binary search.
8. Output a lune λ, if none of the subproblems found a point inside λ.

Clarkson et al. [10] have described an algorithm that partitions the arrangement of r spheres into $O(r^3\beta(r))$ constant size cells in time $O(r^3\beta(r)\log r)$. Therefore Step 3 can be accomplished in time $O(r^3\beta(r)\log r)$ time. Since $r = O(1)$, S_τ, $\mathscr{L}_\tau$ for all cells can be computed in linear time. Finally, the subproblems for 1-dimensional cells can be solved in $O((m+n)\log m)$ time. Therefore, if $T(m, n)$ denotes the maximum expected running time of the algorithm for m points and n lunes, we obtain the following recurrence

$$T(m, n) = \begin{cases} O(n^{1+\varepsilon}) & \text{if } m \leq n^{1/3}, \\ O((m+n)\log n) + \sum_{\tau \in \mathscr{A}(\mathscr{C})} T(m_\tau, n_\tau) & \text{if } m > n^{1/3} \end{cases} \tag{3.1}$$

where $\sum_{\tau \in \mathscr{A}(\mathscr{C})} m_\tau \leq m$. By methods of [9], one can show that the expected values

$$E\left[\sum_{\tau \in \mathscr{A}(\mathscr{C})} n_\tau\right] = O(nr^2\beta(r))$$

and

$$E\left[\sum_{\tau \in \mathscr{A}(\mathscr{C})} m_\tau n_\tau^{2/3}\right] = O\left(m\frac{n^{2/3}}{r^{2/3}}\right).$$

It is known that the solution of the above recurrence is $O(m^{3/4+\varepsilon}n^{3/4} + n^{1+\varepsilon} + m\log^2 n)$; see [1] for a proof. Hence, we obtain the following theorem.

Theorem 3.3. *Given a set S of m points in $\mathbb{R}^3$ and a set $\mathscr{L}$ of n lunes, one can compute, in randomized expected time $O(m^{3/4+\varepsilon}n^{3/4} + n^{1+\varepsilon} + m\log^2 n)$, the lunes of $\mathscr{L}$ that do not contain any point of S in their interior.*

Remark 3.4. As earlier, we can modify this algorithm to determine, in randomized expected time $O(m^{3/4}n^{3/4+\varepsilon}n^{1+\varepsilon})$, the lunes of $\mathscr{L}$ that contain less than k points of S in their interior.

Going back to the problem of computing RNG(S), since we have n points and $O(n)$ lunes, the above theorem immediately implies the following theorem.

Theorem 3.5. *Given a set S of n points in $\mathbb{R}^3$ in general position, one can compute its relative neighborhood graph by a randomized algorithm, whose expected running time is $O(n^{3/2+\varepsilon})$.*

Remark 3.6. A r-element subset $\mathscr{C}$ with properties needed in Step 2 of the above algorithm can be computed deterministically in time $O(n)$ by an algorithm of [18], which implies that RNG(S) can be computed deterministically within the same time bound as mentioned in the above theorem.

The algorithm for computing the set of empty lunes can be extended to higher dimensions as in [1, 3]. The result is that in $\mathbb{R}^d$, one can find the lunes that do not

contain any points of S in time

$$O(m^{d/(d+1)}n^{d/(d+1)+\varepsilon} + m\log^2 n + n^{1+\varepsilon}),$$

which yields

Theorem 3.7. *Given a set S of n points in $\mathbb{R}^d$ in general position, one can compute* RNG(S) *in time* $O(n^{2(1-1/(d+1))+\varepsilon})$.

4. Computing RNG: the general case

If the points of S are not in general position, a point can have several closest neighbors. Consequently, we cannot afford to compute all bi-chromatic closest neighbors for each point $z \in P \cup Q$. Instead we compute all symmetric bi-chromatic closest neighbor pairs of (P, Q) by replacing step II of the previous algorithm with the following step.

II′. For each pair $(P, Q) \in \mathcal{F}$ do the following:

II′.a For each point $z \in P \cup Q$, compute the distance between z and its bi-chromatic closest neighbor; let δ_z denote this distance.

II′.b For each $p \in P$, determine the points $q \in Q$ such that $d(p, q) = \delta_p$ and $\delta_q = \delta_p$. Let $E_{P,Q}$ denote the resulting set of pairs, i.e.,

$$E_{P,Q} = \{(p, q) \in P \times Q \mid d(p, q) = \delta_q = \delta_p\}.$$

In view of Lemma 2.1, RNG(S) is a subgraph of $G = (S, \bigcup_{(P,Q)\in\mathcal{F}} E_{P,Q})$. By Corollary 2.3, G has only $O(n^{4/3})$ edges. After having computed G, we can use the same algorithm as earlier to prune the edges of G that are not in RNG(S). Since there are $O(n^{4/3})$ lunes and n points, by Theorem 3.3, step III will require $O(n^{7/4+\varepsilon})$ time.

Going back to step II′, δ_z for every $z \in P \cup Q$ can be computed in time $O(n^{3/2}\log n)$ as earlier. So we only have to show how to compute the set $E_{P,Q}$ assuming that we know the value of δ_z for all $z \in P \cup Q$.

Let $Q_\beta = \{q \in Q \mid \delta_q = \beta\}$. We process each of Q_β a follows. We map Q_β to a set of hyperplanes $\varphi(Q_\beta)$ in $\mathbb{R}^4$, where a point $q = (q_1, q_2, q_3)$ of Q_β is mapped to the hyperplane

$$\varphi(q)\colon x_4 = 2q_1x_1 + 2q_2x_2 + 2q_3x_3 - (q_1^2 + q_2^2 + q_3^2). \tag{4.1}$$

We process the upper envelope of $\varphi(Q_\beta)$, in time $O(m^{2+\varepsilon})$, so that, for a query point ξ, one can determine all k hyperplanes of $\varphi(Q_\beta)$ containing ξ in time $O(\log n + k)$. This can be easily done by modifying the algorithm of Clarkson [7].

For every point $p \in P$, we first determine whether the set Q_{δ_p} is empty. If not, we report the points $q \in Q_{\delta_p}$ for which $\bar{p} = (p_1, p_2, p_3, p_1^2 + p_2^2 + p_3^2 - \delta_p^2)$ lies on the hyperplane $\varphi(q)$. Since $d(p, q) \geq \delta_p$, for all $q \in Q_{\delta_p}$, none of the hyperplanes

of $\varphi(Q_{\delta_p})$ lie above $\bar{p}$, i.e., $\bar{p}$ lies in the upper envelope of Q_{δ_p}. Therefore, we can report all k points q with $d(p, q) = \delta_p$, in time $O(\log n + k)$, using the above structure. Let K be the total number of points returned by the above procedure, then the total running time is $O(m^{2+\varepsilon} + n \log m + K)$. This can be improved to $O(mn^{1/2+\varepsilon} + n^{1+\varepsilon} + K)$ using the batching technique similar to the one used in the first algorithm of the previous section. By Corollary 2.3, $K = O(m^{2/3}n^{2/3} + m + n)$. Consequently, $E_{P,Q}$ can be computed in time $O(mn^{1/2+\varepsilon}n^{1+\varepsilon})$. Repeating this procedure over all pairs $(P, Q) \in \mathcal{F}$, we obtain the graph G, in time $O(n^{3/2+\varepsilon})$. Hence, we can conclude

Theorem 4.1. *Given a set S of arbitrary n points in $\mathbb{R}^3$, we can construct* RNG(S) *in time* $O(n^{7/4+\varepsilon})$.

Remark 4.2. (i) The time required to construct the graph G can be further improved to $O(n^{4/3+\varepsilon})$ by using the random sampling technique as for step III in the previous section, but for our purpose this algorithm is good enough.

(ii) If the upper bound on the number of bi-chromatic closest pairs between two sets of points, with n points in each set, can be improved to τ_n, one can compute RNG(S) in time $O(\tau_n^{3/4}n^{3/4+\varepsilon})$.

5. k-Relative neighborhood graphs

The k-relative neighborhood graph of S, denoted k-RNG(S), is a generalization of RNG(S). In particular, (p, q) is an edge of k-RNG(S) if $\lambda(p, q)$ contains less than k points of S. For $k = O(1)$, Chang et al [4] have proposed an $O(n^2)$ algorithm to compute k-RNG(S) in $\mathbb{R}^2$, which has been improved by Su and Chang to $O(n^{5/3} \log n)$ [21]. It is known that *Euclidean bottleneck matching* problem, i.e., given a set S of points find an Euclidean matching that minimizes the longest edge, can be solved efficiently using k-RNG(S) [4]. In fact some other Euclidean bottleneck problems can also be reduced to computing k-relative neighborhood graphs.

In this section we present efficient algorithms for computing the k-relative neighborhood graph of a set of points. Let us begin by an easy consequence of Theorem 2.4.

Theorem 3.5. *Given a set S of n points in $\mathbb{R}^2$ (resp. $\mathbb{R}^3$), the k-*RNG(S) *has* $O(nk)$ *(resp.* $O(n^{4/3}k^{2/3})$*) edges.*

Proof. The proof is based on a method due to Clarkson [9]. Rather than explaining the general theory from which the result directly follows, we just give the specific application. Let M_k denote the number of edges of k-RNG(S). Choose a random r-element subset $R \subseteq S$, where $r = \lfloor n/k \rfloor$ and each r-element

subset of S is chosen with equal probability. Let $\mathbf{P}_{p,q}$ denote the probability that an edge $(p, q) \in k\text{-RNG}(S)$ is an edge of RNG(R). An edge (p, q) of k-RNG(S) will be an edge of RNG(R) if the following two conditions are satisfied.

(i) Both p, q are chosen into R, and

(ii) none of the (at most k) points of $S \cap \lambda(p, q)$ belong to R.

Therefore

$$\mathbf{P}_{p,q} \geq \binom{n-k-2}{r-2} \Big/ \binom{n}{r} \geq \frac{c}{k^2},$$

where c is some constant. The expected number of edges in RNG(R) is thus

$$E[|\text{RNG}(R)|] \geq \sum_{(p,q)\in k\text{-RNG}(S)} \mathbf{P}_{p,q} \geq \frac{c}{k^2} M_k.$$

But RNG(R) can have only $O(r^{4/3})$ edges in $\mathbb{R}^3$ (cf. Theorem 2.4), which implies $M_k = O(n^{4/3}k^{2/3})$.

A similar argument shows that $M_k = O(nk)$ in $\mathbb{R}^2$. □

In this section, we obtain efficient algorithms for computing k-RNG(S), for $k = O(1)$, by modifying the algorithms described in the previous sections. We shall describe only the main idea.

For a point p, let $\Phi_k(p, Q) \subseteq Q$ denote the set of points q such that $d(p, q)$ is less than or equal to the distance between p and its kth closest neighbor in Q. Following the same argument as in Lemma 2.1, one can show that if P, Q are well separated, then k-RNG($P \cup Q$) has a cross edge (p, q) only if $q \in \Phi_k(p, Q)$ and $p \in \Phi_k(q, p)$. Hence, we can modify the algorihtm of Section 3 as follows.

I. Compute the family $\mathcal{F}$ of well separated pairs of subsets of S, as described above.

II. For each pair $(P, Q) \in \mathcal{F}$, compute $\Phi_k(p, Q)$, for every $p \in P$, and $\Phi_k(q, P)$, for every $q \in Q$. Let $E_{P,Q} \subset P \times Q$ be the set of edges (p, q) such that $p \in \Phi_k(q, P)$ and vice-versa. Let $G = (S, \bigcup_{(P,Q)\in\mathcal{F}} E_{P,Q})$.

III. Throw away an edge (p, q) of G if $\lambda(p, q)$ contains $\geq k$ points of S.

Agarwal and Matoušek have shown that after $O(s^{1+\varepsilon})$ preprocessing $(n \leq s \leq n^2)$, one can compute $\Phi_k(p, Q)$, in time $O(n^{1+\varepsilon}/\sqrt{s})$ [2]. Setting $s = m^{2/3}n^{2/3} + m + n$, step II, for a fixed pair (P, Q), can be performed in time $O(m^{2/3}n^{2/3+\varepsilon} + m + n)$ (recall that $k = O(1)$). Thus, the total time spent in steps I and II is $O(n^{4/3+\varepsilon})$. Finally in view of Remark 3.4, the edges of G that are not in k-RNG(S) can be determined in time $O(n^{3/2+\varepsilon})$. Hence, if points of S are in general position, k-RNG(S) can be computed in time $O(n^{3/2+\varepsilon})$. As in Section 3, the same algorithm can compute k-RNG(S) in $\mathbb{R}^d$ in time $O(n^{2(1-1/(d+1))+\varepsilon})$.

Using Theorem 5.1, and an appropriate modification of the algorithm of Section 4, we can compute k-RNG(S) in time $O(n^{4/3+\varepsilon})$ (resp. $O(n^{7/4+\varepsilon})$) for $d=2$ (resp. $d=3$).

Hence, we can conclude the following theorem.

Theorem 5.2. *Given a set S of n points in $\mathbb{R}^d$ in general position and a fixed constant k, one can compute k-RNG(S) in time $O(n^{2(1-1/(d+1))+\varepsilon})$. If points of S are arbitrary, k-RNG(S) can be computed in time $O(n^{4/3+\varepsilon})$ (resp. $O(n^{7/4+\varepsilon})$) for $d=2$ (resp. $d=3$).*

6. Conclusion

In this paper, we describe efficient algorithms for computing relative neighborhood graphs and k-relative neighborhood graphs in $\mathbb{R}^d$. Our algorithms work for larger values of k too, but analysis of the running time becomes complicated. Moreover, in most of the applications k is some fixed constant.

We conclude by mentioning an open problem: "What is the number of bichromatic closest pairs between a set of m points and another set of n points in $\mathbb{R}^3$?"

References

[1] P. Agarwal, H. Edelsbrunner, O. Schwarzkopf and E. Welzl, Euclidean minimum spanning trees and bichromatic closest pairs, Discrete Comput. Geom. 6 (1991) 407–422.

[2] P. Agarwal and J. Matoušek, Ray shooting and parametric search, in: Proc. 24th Symp. Theory of Computing, 1992, to appear.

[3] P. Agarwal, J. Matoušek and S. Suri. Farthest neighbors, maximum spanning trees and related problems in higher dimensions, Comput. Geom. Theory Appl. 1 (1992) 189–201.

[4] M. Chang, C. Tang and R. Lee, Solving the Euclidean bottleneck matching problem by k-relative neighborhood graph, Algorithmica, to appear.

[5] B. Chazelle and J. Friedman, A deterministic view of random sampling and its use in geometry, Combinatorica 10 (1990) 229–249.

[6] B. Chazelle, M. Sharir and E. Welzl, Quasi-optimal upper bounds for simplex range searching and new zone theorems. Proceedings 6th Annual Symposium on Computational Geometry (1990) 23–33.

[7] K. Clarkson, A randomized algorithm for closest-point queries, SIAM J. Comput. 17 (1988) 830–847.

[8] K. Clarkson, New applications of random sampling in computational geometry, Discrete Comput. Geom 2 (1987) 195–222.

[9] K. Clarkson and P. Shor, New applications of random sampling in computational geometry II, Discret Gomput. Geom 4 (1989) 387–421.

[10] K. Clarkson, H. Edelsbrunner, L. Guibas, M. Sharir and E. Welzl, Combinatorial complexity bounds for arrangements of curves and spheres, Discrete Comput. Geom 5 (1990) 99–160.

[11] H. Edelsbrunner and M. Sharir, A hyperplane incidence problem with applications to counting distances, in: P. Gritzmann and B. Sturmfels, eds., Applied Geometry & Discrete Mathematics: The Victor Klee's Festschrift (AMS Press, Providence, RI, 1991) 253–263.

[12] M. Ichino and J. Sklansky, The relative neighborhood graph for mixed feature variables, Pattern Recognition 18 (1985) 161–167.
[13] J. Jaromczyk and M. Kowaluk, A note on relative neighborhood graphs, Proceedings 3rd Annual Symposium on Computational Geometry (1987) 233–241.
[14] J. Jaromczyk and M. Kowaluk, Constructing the relative neighborhood graph in 3-dimensional Euclidean space, Discrete Appl. Math. 31 (1991) 181–191.
[15] J. Jaromczky, M. Kowaluk and F. Yao, An optimal algorithm for constructing β-skeltons in L_p metric, SIAM J. Comput., to appear.
[16] J. Katajainen, The region approach for computing relative neighborhood graphs in the L_p metric, Computing 40 (1988) 147–161.
[17] J. Katajainen and O. Nevalainen, Computing relative neighborhood graphs in the plane, Pattern Recognition 19 (1986) 221–228.
[18] J. Matoušek, Approximations and optimal geometric divide-and-conquer, Proc. 23. ACM Symposium on Theory of Computing (1991) 506–511.
[19] J. O'Rourke, Computing the relative neighborhood graph in L_1 and L_∞ metrics, Pattern Recognition 14 (1982) 45–55.
[20] T. Su and R. Chang, Computing the constrained relative neighborhood graphs and constrained gabriel graphs in Euclidean plane, Pattern Recognition 24 (1991) 221–230.
[21] T. Su and R. Chang, Computing the k-relative neighborhood graphs in the Euclidean plane. Pattern Recognition 24 (1991) 231–239.
[22] K. Supowit, The relative neighborhood graph, with an application to minimum spanning trees, J. ACM 30 (1983) 428–448.
[23] G. Toussaint, The relative neighborhood graph of a finite planar set, Pattern Recognition 12 (1980) 261–268.
[24] G. Toussaint, Pattern recognition and geometric complexity, Proc. 5th International Conference on Pattern Recognition (1980) 1–24.
[25] R. Urquhart, Some properties of the planar Euclidean relative neighborhood graph, Pattern Recognition Lett. 1 (1983) 317–322.
[26] A. Yao, On constructing minimum spanning trees in k-dimensional spaces and related problems, SIAM J. Comput. 11 (1982) 721–736.